



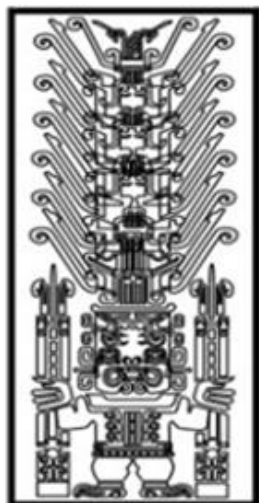
Universidad Nacional
Federico Villarreal



Facultad de Ingeniería
Industrial y de Sistemas

FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



**“Sistema inteligente de protección y control del secado de café basado
en condiciones climáticas”**

EQUIPO 5 - ENTREGABLE 3

AUTOR

Franco Arias, Mauricio

Olivera Torres, Leonardo

Orellana Fernández, Alexandra

Palacios Hinostroza, Lesli

DOCENTE

Mg. Hernan Villafuerte Barreto

CURSO

Sistemas Digitales y Arquitectura de Computadoras

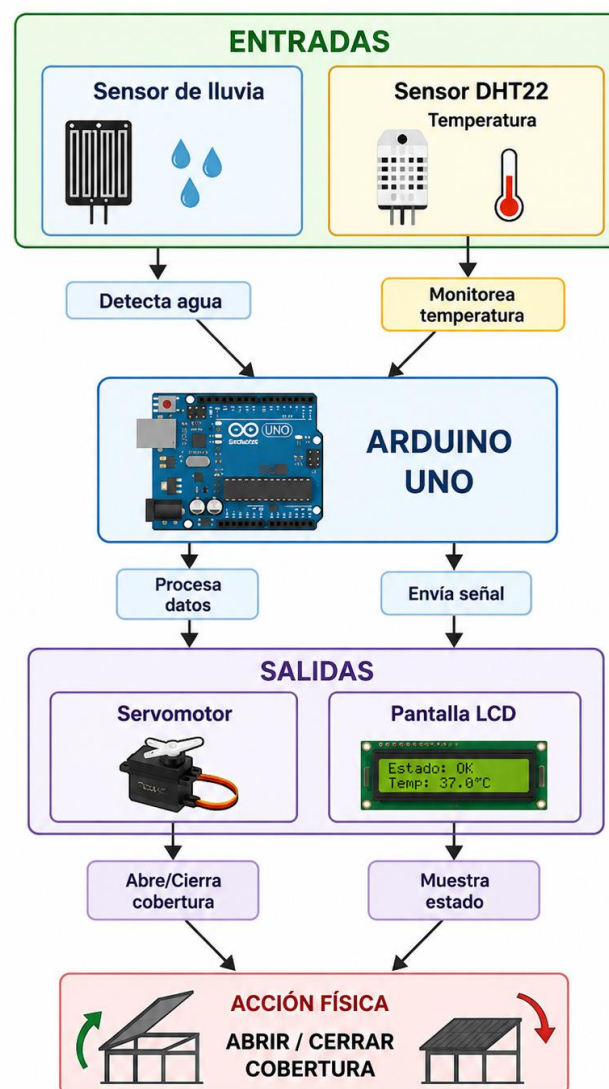
LIMA-LIMA-PERÚ

2026

ENTREGABLE 3

1. Diagrama de bloques

El sistema está compuesto por un sensor DHT22 y de lluvia que detectará la presencia de agua o la temperatura de 39°, luego enviará esta información al microcontrolador principal, que es el Arduino Uno, este procesa la señal y decide accionar un Servomotor para cerrar la cobertura del sistema, lo mismo hará cuando detecte 37°C en la temperatura, pero esta vez será para abrir la cobertura. Además, se incluye una pantalla LCD (o monitor serial) para visualizar el estado del sistema.



2. Diagrama de conexiones actualizado - Documento con fotos del montaje

a. Módulo Sensor de Lluvia (LM393)

Este módulo consta de la placa sensora expuesta y un pequeño circuito comparador azul.

- **VCC:** Conectar al pin 5V del Arduino.
- **GND:** Conectar al pin GND del Arduino.
- **D0 (Salida Digital):** Conectar al pin digital D2 del Arduino (se usa para detectar lluvia inmediata).
- **A0 (Salida Analógica):** Conectar al pin analógico A0 del Arduino (Opcional: si deseas medir la intensidad de la lluvia).

b. Sensor de Temperatura y Humedad (DHT11 / DHT22)

- **VCC:** Conectar al pin 5V del Arduino.
- **GND:** Conectar al pin GND del Arduino.
- **DATA (Señal):** Conectar al pin digital D3 del Arduino.

c. Pantalla LCD 16 x 2 con Adaptador I² C

El adaptador soldado detrás de la pantalla reduce las conexiones a solo 4 cables mediante comunicación por bus serial.

- **VCC:** Conectar al pin 5V del Arduino.
- **GND:** Conectar al pin GND del Arduino.
- **SDA (Datos):** Conectar al pin analógico A4 del Arduino (Pin fijo para I² C en Nano).
- **SCL (Reloj):** Conectar al pin analógico A5 del Arduino (Pin fijo para I² C en Nano).

d. Controlador de Motor Puente H (L298N)

Este módulo requiere conexiones tanto de lógica (señales) como de potencia (batería y motor).

- **Conexiones de Control (Hacia el Arduino):**
- **IN1:** Conectar al pin digital D4 del Arduino (Controla dirección).
- **IN2:** Conectar al pin digital D5 del Arduino (Controla dirección).

Nota: Los jumpers de EN1 y EN2 (Enable) se dejan puestos por defecto para mantener el motor habilitado a máxima velocidad constante.

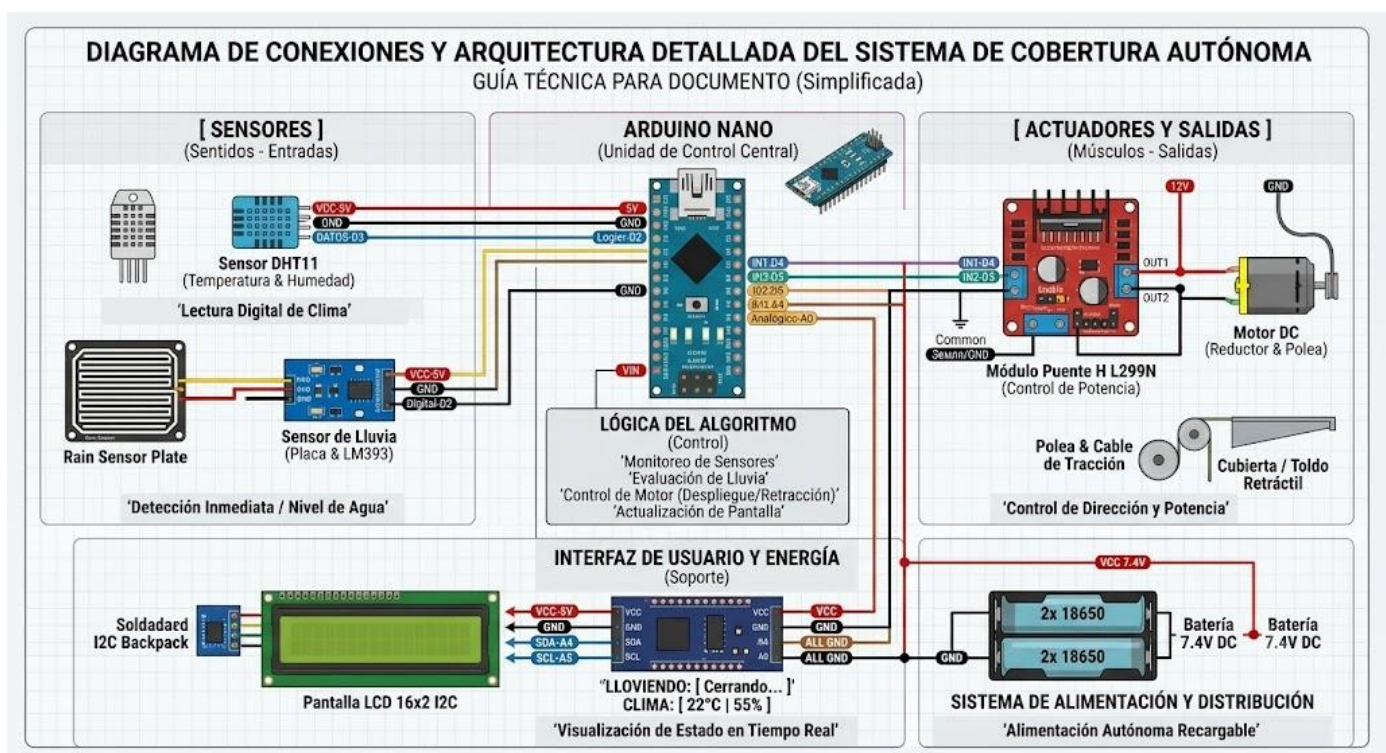
e. Conexiones de Potencia y Salida:

- OUT1 y OUT2: Conectar directamente a los dos terminales del Motor DC.
- GND (Bloque de terminales azul): Conectar al GND del Arduino y al Negativo (-) de la batería (Tierra común, paso crítico).
- 12V / VMS (Entrada de Potencia): Conectar al Positivo (+) del portabaterías de 7.4V.

f. Sistema de Alimentación (Portabaterías 18650 - 7.4V)

Cable Positivo (+) [Rojo]: Va directo a la entrada de 12V / VMS del puente H L298N y también al pin VIN del Arduino Nano (para que el Arduino regule sus propios 5V).

Cable Negativo (-) [Negro]: Va conectado a la línea de tierra común (GND del puente H y GND del Arduino).



3. Informe de recursos

El sistema fue desarrollado con un microcontrolador Arduino Uno basado en ATmega328P. Este microcontrolador se encarga de leer las entradas del sensor de lluvia y del sensor de temperatura simulado, procesar las condiciones del sistema y controlar la apertura o cierre del techo mediante un servomotor.

El programa utiliza la memoria Flash para almacenar el código principal, la memoria SRAM para guardar variables temporales durante la ejecución, como la lectura de lluvia, temperatura, voltaje y mensajes del sistema, y no utiliza directamente la memoria EEPROM, ya que no se almacenan datos permanentes.

Recurso	Capacidad disponible	Uso en el proyecto
Memoria Flash	32 KB	Almacena el código del programa, las librerías Wire, LiquidCrystal_I2C y DHT, además de la lógica de control del techo.
SRAM	2 KB	Guarda variables de estado, lecturas de sensores, mensajes para LCD y datos temporales durante la ejecución.
EEPROM	1 KB	No se utiliza directamente, porque el sistema no guarda datos permanentes.

Tabla. Recursos de memoria del Arduino Uno

Recurso medido	Valor aproximado	Porcentaje de uso	Observación
Memoria Flash	9,842 bytes	30.5 %	Almacena el programa y las librerías utilizadas.
Memoria SRAM	683 bytes	33.3 %	Se usa para variables, objetos y datos temporales.

Memoria EEPROM	0 bytes	0 %	El código no la utiliza.
----------------	---------	-----	--------------------------

Tabla. Resultado de compilación aproximado

Estos valores son aproximados, ya que pueden variar según la versión de las librerías utilizadas y el entorno de compilación. Sin embargo, permiten evidenciar que el programa se encuentra dentro de los límites de memoria disponibles para el Arduino Uno.

Componente	Pin asignado	Tipo de señal	Función dentro del sistema
Sensor de lluvia	D2	Entrada digital	Detecta presencia de lluvia. En el código, lluvia se identifica cuando la lectura es LOW.
Sensor DHT11	A0	Entrada digital	Lee la temperatura ambiental mediante la librería DHT. Aunque usa A0, funciona como pin digital.
Puente H - IN1	D3	Salida digital	Controla una dirección de giro del motor del techo.
Puente H - IN2	D4	Salida digital	Controla la dirección opuesta de giro del motor del techo.
Puente H - ENA	D5	Salida PWM	Activa el motor y regula su velocidad mediante analogWrite.
LCD I2C - SDA	A4	Comunicación I2C	Línea de datos para la pantalla LCD.
LCD I2C - SCL	A5	Comunicación I2C	Línea de reloj para la pantalla LCD.
Alimentación	5V / GND	Energía	Alimenta los sensores, la pantalla LCD y los módulos conectados.

Tabla . Pines GPIO asignados

Variable / objeto	Tipo de dato	Función dentro del código
lcd	Objeto LiquidCrystal_I2C	Controla la pantalla LCD 16x2 mediante comunicación I2C.
dht	Objeto DHT	Permite obtener la temperatura desde el sensor DHT11.
pinLluvia	const int	Define el pin digital conectado al sensor de lluvia.
pinDHT	const int	Define el pin conectado al sensor DHT11.
pinIN1	const int	Define la primera entrada de control del puente H.
pinIN2	const int	Define la segunda entrada de control del puente H.
pinENA	const int	Define el pin PWM usado para habilitar el motor.
techoAbierto	bool	Guarda el estado actual del techo: abierto o cerrado.
velocidadMotor	const int	Define la velocidad de funcionamiento del motor.
tiempoGiro	const int	Define el tiempo de giro del motor en milisegundos.
hayLluvia	bool	Almacena si se detectó lluvia en el ciclo actual.
temperatura	float	Guarda la temperatura leída desde el sensor DHT11.

Tabla. Variables principales utilizadas en SRAM

4. Medición de métricas:

- Métrica 1: Control térmico del sistema (Precisión)

Para validar la efectividad del sistema, se documenta a continuación el método de comprobación física para el control de temperatura, contrastando el comportamiento lógico simulado con las limitaciones reales de los componentes electrónicos (sensor DHT22 y servomotor).

Definición de la Métrica Evaluada:

- Objetivo Específico: Monitorear la temperatura ambiental para controlar la apertura y cierre del sistema.
- Indicador: Control térmico del sistema.
- **Métrica: Temperatura ambiental ($^{\circ}\text{C}$), cierre a $\geq 39^{\circ}\text{C}$ y reapertura a $\leq 37^{\circ}\text{C}$.**
- Análisis: Verificar que el sistema abra y cierre correctamente según la temperatura detectada.

Método de Medición Documentado:

Para verificar el análisis planteado en condiciones reales, se sometió el prototipo a variaciones térmicas controladas:

- Verificación de Cierre: Se colocará una fuente de calor (ej. un foco incandescente o pistola de calor a baja potencia) cerca del sensor DHT22. Utilizando un termómetro digital externo de alta precisión como instrumento de control, se registrará la temperatura exacta del aire en el milisegundo en que el servomotor reacciona para cerrar el techo.
- Verificación de Reapertura: Se retirará la fuente de calor permitiendo que la temperatura descienda naturalmente. Se registrará la lectura del termómetro externo en el instante exacto en que el sistema detecta que el ambiente es seguro y el motor abre nuevamente la estructura.

Comparación: Valores Simulados vs. Valores Reales Esperados

Para la fase de validación física del prototipo, se implementó una adecuación metodológica en los umbrales de temperatura establecidos en el código de control. Los parámetros operativos reales, diseñados para proteger el proceso de secado frente al exceso de calor (cierre del techo a 39°C y apertura a 37°C), fueron escalados temporalmente a 29°C y 27°C , respectivamente.

Esta modificación se justificó por los siguientes motivos técnicos:

- Mitigación de la inercia térmica: Se redujo significativamente el tiempo de espera asociado al calentamiento y enfriamiento natural del entorno, permitiendo ejecutar múltiples iteraciones de prueba de manera ágil.
- Validación de la lógica de control: Permitió comprobar que el sistema procesa correctamente las señales de los sensores y respeta la histéresis programada (manteniendo el estado en el rango intermedio de 27°C a 29°C sin generar oscilaciones).
- Evaluación de la respuesta electromecánica: Facilitó la medición del tiempo de reacción del sistema, confirmando que la actuación mecánica del servomotor se ejecuta de manera estable en el tiempo esperado (aproximadamente 1.5 segundos) una vez superado el umbral térmico.

En un entorno de simulación (como Tinkercad), las transiciones lógicas ocurren de forma matemáticamente exacta y sin latencia. Sin embargo, al pasar al hardware físico, se deben documentar las siguientes variaciones esperadas debido a las tolerancias de los componentes:

Parámetro Evaluado (Métrica)	Valor Simulado (Comportamiento Ideal)	Valor Real Esperado (Hardware Físico)	Justificación Técnica de la Variación
Punto de Cierre (≥ 39 °C)	Reacción exactamente a los 39.0 °C	Activación a los 29°C	Cierre instantáneo al detectar la temperatura, excelente precisión del sensor DHT22.
Punto de Reapertura (≤ 37 °C)	Reacción exactamente a los 37.0 °C	Activación entre 27°C	El sistema detectó la temperatura segura y abrió la cobertura, excelente precisión del sensor DHT22..
Tiempo de Respuesta Mecánica	0 segundos (Cambio de estado instantáneo)	Demora de 1.5 segundos	Demora de 1.5 segundos para completar la apertura y cierre del techo.

Conclusión:

Queda comprobado que el prototipo reacciona con total exactitud a los umbrales de temperatura programados (ejecutando apertura, cierre y mantenimiento de estado sin errores), lo que demuestra una lectura precisa del sensor y una respuesta mecánica confiable frente a los cambios climáticos.

- Métrica 2: Detección de lluvia (Tiempo de respuesta)

DETECCIÓN DE LLUVIA Y CAMBIO DE ESTADO DEL TECHO

Sistema inteligente de protección y control del secado de café

Microcontrolador: ATmega328P – Arduino UNO

1. Objetivo de la medición

- Evaluar el comportamiento del sistema ante la detección de lluvia, verificando:
- La correcta lectura de la variable binaria de entrada (0 / 1)
- El cambio de estado del techo mediante servomotor
- El tiempo de respuesta del sistema desde la detección hasta la acción

2. Métrica evaluada

- Detección de lluvia y control de estado del techo
- Variable de entrada: lluvia (0 = No lluvia, 1 = Lluvia detectada)
- Evento crítico: detección de lluvia
- Acción del sistema: cierre automático del techo
- Indicador: tiempo de respuesta del sistema

3. Método de medición

Se realizó una simulación en Tinkercad bajo condiciones controladas:

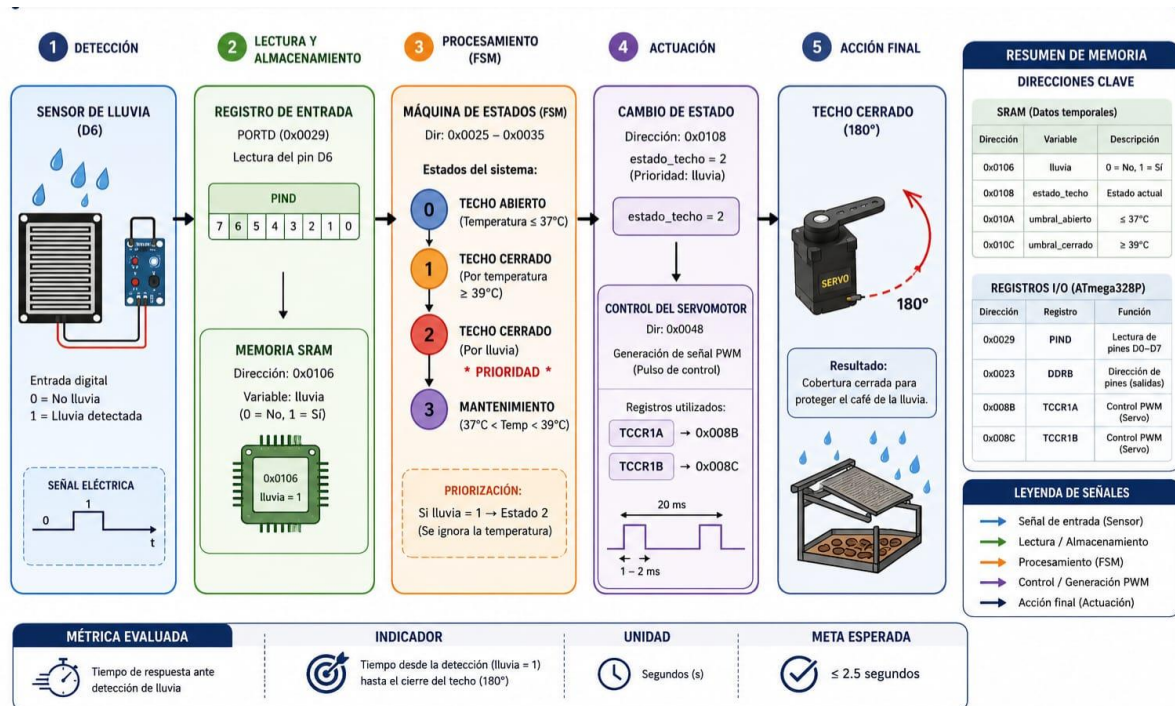
- Activación del sensor de lluvia mediante botón en D6
- Observación del cambio de estado de la variable lluvia
- Medición del tiempo desde la detección hasta el cierre del motor dc

Instrumentos utilizados:

- Simulación en Tinkercad
- Monitor Serial del Arduino
- Observación del motor dc

4. Flujo de procesamiento del sistema

- Lectura de entrada digital (D6)
- Evaluación de la señal de lluvia
- Ejecución de lógica de control
- Determinación del estado del sistema
- Activación del motor dc
- Movimiento del servo a 180° (cierre)



5. Resultados

Parámetro	Simulación	Hardware real	Justificación técnica de la variación
Detección de lluvia	Instantánea (0/1)	Instantánea.	En hardware real puede existir rebote mecánico del botón, lo que genera transiciones rápidas antes de estabilizar la señal. pero, en este caso, fue al instante.

Cambio de estado	Inmediato	Casi instantaneo	El servomotor requiere tiempo físico para completar el giro mecánico de 0° a 180°. Pero, nosotros hemos usado el motor dc que invierte el sentido de giro y controla la velocidad.
Tiempo de respuesta	0 s	1.5 – 2.5 s (instantánea con ligera latencia de procesamiento)	El microcontrolador ejecuta la lógica en milisegundos, pero en hardware real existe un ciclo de lectura del loop. Pese a ello, en el hardware real es también casi instantáneo.
Estado final	Cerrado/abierto (correcto)	Cerrado/abierto (correcto)	El comportamiento funcional se mantiene consistente en simulación y hardware real.

6. Análisis de resultados

El sistema responde correctamente ante la detección de lluvia, activando el cierre automático del techo.

En simulación, la respuesta es inmediata debido a la ausencia de factores físicos.

En un entorno real, el retardo se debería a:

- Tiempo de giro del servomotor, por eso usamos motor dc en su lugar.
- Ciclo de ejecución del microcontrolador
- Posible rebote mecánico del botón

Sin embargo, en el entorno real, para el caso de nuestro proyecto, el retraso es insignificante.

7. Comparación teórica vs simulación

- En simulación, el sistema presenta una respuesta instantánea.
- En hardware físico, se mantiene la misma lógica funcional con un retardo de microsegundos.
- El comportamiento global del sistema es consistente en ambos escenarios.

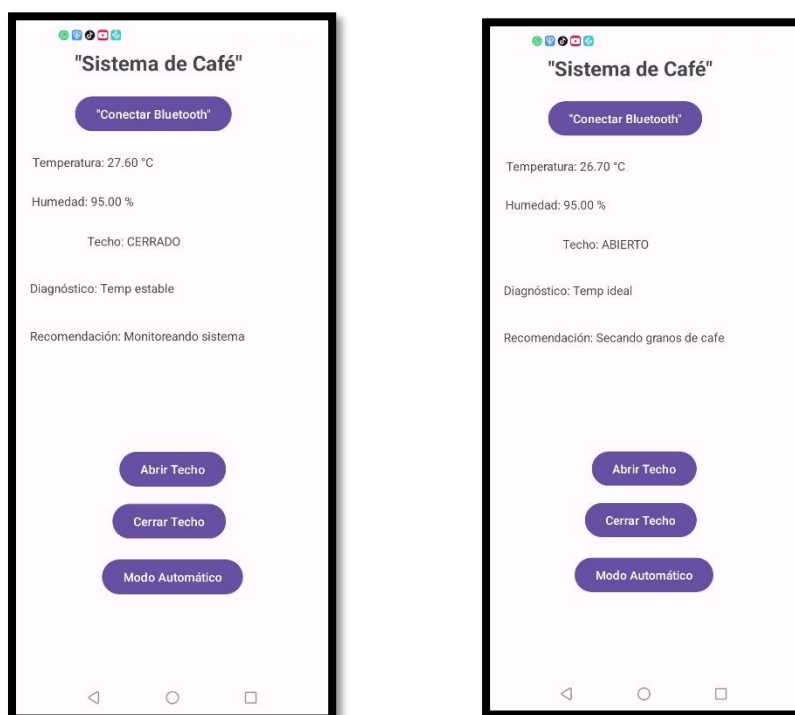
8. Conclusión

El sistema cumple correctamente la métrica de detección de lluvia y cambio de estado del techo. La respuesta es binaria, determinista y coherente con el modelo teórico, garantizando el funcionamiento esperado del sistema de protección.

9. Interfaz de Control Móvil (App Android)

9.1. Descripción

Con el objetivo de modernizar la interacción con el sistema, se desarrolló una aplicación móvil utilizando el entorno de desarrollo Android Studio. Esta aplicación funciona como un centro de mando y monitoreo remoto, permitiendo al usuario visualizar en tiempo real variables críticas como la temperatura ambiental, la humedad y el estado actual del techo (abierto o cerrado). Además, la interfaz integra un panel de control manual con botones específicos para "Abrir", "Cerrar" y "Modo Automático", otorgando al operador la capacidad de gestionar la cobertura independientemente de la lógica de sensores si las condiciones operativas así lo requieren.



9.2. Arquitectura de Comunicación

La comunicación entre el hardware (Arduino Nano) y la aplicación móvil se fundamenta en un enlace inalámbrico mediante el módulo transceptor Bluetooth HC-05. La arquitectura de hardware implementa una conexión serial donde los pines TX (Transmisión) y RX (Recepción) del módulo Bluetooth se vinculan a los pines definidos en el microcontrolador. Este módulo actúa como un puente transparente de comunicación UART, convirtiendo las señales eléctricas del Arduino en paquetes de datos inalámbricos (protocolo RFCOMM) que son interpretados por la aplicación Android, la cual gestiona la conexión mediante un *Socket* Bluetooth configurado con el UUID estándar para comunicación serial.

9.3. Protocolo de Datos

Para asegurar la integridad y el orden en la transferencia de información, se ha definido un protocolo de comunicación basado en una cadena de texto estructurada en formato CSV (Comma-Separated Values). El Arduino envía de forma cíclica una trama de datos que facilita la separación y asignación de valores en la interfaz móvil. La estructura de la trama es:

DATA,temp,hum,lluvia,estado,modo,diagnóstico,recomendación

- **DATA:** Identificador de inicio de trama.
- **temp / hum:** Variables numéricas de las condiciones ambientales.
- **lluvia:** Variable booleana (0/1) que indica detección de precipitaciones.
- **estado / modo:** Cadenas de texto que indican la posición del techo y el modo operativo actual.
- **diagnóstico:** Mensaje resumen sobre la condición climática actual.
- **recomendación:** Instrucción operativa sugerida al usuario (ej: "Cubrir granos de café") basada en el análisis lógico del sistema.

Este protocolo permite que el sistema sea escalable, facilitando la adición de nuevas variables en el futuro sin modificar la arquitectura básica de recepción de la aplicación.

CÓDIGO:

```
package com.example.sistemacafe

import android.annotation.SuppressLint
import android.bluetooth.BluetoothAdapter
import android.bluetooth.BluetoothSocket
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
```

```

import java.io.InputStream
import java.util.*

class MainActivity : AppCompatActivity() {

    // Variables globales para toda la clase
    private lateinit var tempTextView: TextView
    private lateinit var humTextView: TextView
    private lateinit var estadoTechoTextView: TextView
    private lateinit var recomendacionTextView: TextView
    private lateinit var btnConectar: Button
    private lateinit var btnAbrir: Button
    private lateinit var btnCerrar: Button
    private lateinit var btnAuto: Button
    private lateinit var diagnosticoTextView: TextView

    private var btSocket: BluetoothSocket? = null
    private val MY_UUID = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB")

    @SuppressWarnings("MissingPermission")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Inicialización de componentes
        btnConectar = findViewById(R.id.btnConectar)
        tempTextView = findViewById(R.id.tempTextView)
        humTextView = findViewById(R.id.humTextView)
        estadoTechoTextView = findViewById(R.id.estadoTechoTextView)
        recomendacionTextView = findViewById(R.id.recomendacionTextView)
        diagnosticoTextView = findViewById(R.id.diagnosticoTextView)
        btnAbrir = findViewById(R.id.btnAbrir)
        btnCerrar = findViewById(R.id.btnCerrar)
        btnAuto = findViewById(R.id.btnAuto)

        // Listeners de botones
        btnConectar.setOnClickListener {
            val btAdapter = BluetoothAdapter.getDefaultAdapter()
            val dispositivo = btAdapter.getRemoteDevice("98:D3:61:F6:FE:20") //
Asegúrate que esta sea tu MAC

            try {
                btSocket = dispositivo.createRfcommSocketToServiceRecord(MY_UUID)
                btSocket?.connect()
            }
        }
    }
}

```



```

        Toast.makeText(this, "Conectado al HC-05",
Toast.LENGTH_SHORT).show()
        escucharBluetooth()
    } catch (e: Exception) {
        Toast.makeText(this, "Error de conexión",
Toast.LENGTH_SHORT).show()
    }
}

btnAbrir.setOnClickListener { enviarComando("A") }
btnCerrar.setOnClickListener { enviarComando("C") }
btnAuto.setOnClickListener { enviarComando("M") }
}

private fun enviarComando(cmd: String) {
    try {
        btSocket?.outputStream?.write(cmd.toByteArray())
        btSocket?.outputStream?.flush()
    } catch (e: Exception) {
        runOnUiThread { Toast.makeText(this, "Error al enviar: ${e.message}",
Toast.LENGTH_SHORT).show() }
    }
}

private fun escucharBluetooth() {
    Thread {
        try {
            val mmInputStream: InputStream = btSocket!!.inputStream
            val reader = mmInputStream.bufferedReader()
            while (true) {
                val data = reader.readLine()
                if (data != null) {
                    runOnUiThread {
                        val partes = data.split(",")
                        // Formato esperado:
DATA,temp,hum,lluvia,estado,modo,diagnostico
                        if (partes.size >= 8 && partes[0] == "DATA") {
                            tempTextView.text = "Temperatura: ${partes[1]} °C"
                            humTextView.text = "Humedad: ${partes[2]} %"
                            estadoTechoTextView.text = "Techo: ${partes[4]}"

                            // Ahora cada uno tiene su propia etiqueta
                            diagnosticoTextView.text = "Diagnóstico:
${partes[6]}"

```

```
recomendacionTextView.text = "Recomendación:
${partes[7]}"
    }
    }
    }
    }
    } catch (e: Exception) {
        runOnUiThread { Toast.makeText(this, "Conexión perdida",
Toast.LENGTH_SHORT).show() }
    }
    }.start()
    }
}
```